

学号: 19428424



常州大学

课程实习报告

课程名称: Web 项目实习

题 目: 图书管理系统

学生姓名: 吴梓晋

学 院: 计算机与人工智能学院 专 业 班 级: 计算机 199

指导教师: 宦娟 成 绩:

二〇二二年十二月

目录

1. 前言.....	1
1.1 选题背景.....	1
1.2 设计目标.....	1
2. 运行环境及开发工具概述.....	2
2.1 开发及运行环境概述.....	2
3. 系统的需求分析.....	3
3.1 背景和意义.....	3
3.2 可行性分析.....	3
3.3 需求分析.....	3
4. 系统设计.....	5
4.1 界面设计.....	5
4.2 系统总体设计.....	5
4.3 数据库设计.....	6
4.4 系统实体 E-R 图.....	6
4.5 系统数据表设计.....	7
4.6 数据库逻辑设计.....	8
5. 图书管理系统的实现.....	9
5.1 用户注册登录模块.....	9
5.2 出版社管理模块.....	15
5.3 作者管理模块.....	15
5.4 图书管理模块.....	16
5.5 关键代码.....	17
6. 系统的调试与测试.....	22
7. 总结.....	26
参考文献.....	26

图书管理系统

摘 要：该系统是 B/S 模式下时下流行的图书管理系统，是图书馆需要应用的一套 OA 系统，使管理者可以轻松地随时在电脑上，轻松的对图书进行各种管理操作。需完成系统架构设计、数据库设计和图书管理设计。主要功能有：出版社管理、图书管理、作者管理等功能。

本系统在分析图书管理需求的基础上，在 Windows 10 操作系统下，选择了 PyCharm 开发环境、MySQL 数据库和 Django 和 Bootstrap 框架技术，并阐述了 python, Django 等技术的工作原理及其关系，实现了工作台、用户管理、出版社管理、作者管理和图书管理等后台模块开发。后台数据库选用 MySQL 数据库，并利用 ORM 技术完成对数据库的封装映射，实现了数据查询、修改、增加、删除等功能。前端框架使用 Bootstrap 框架，他很好的整合了 html, CSS 和 JavaScript 等前端语言，是开发者一个很好的选择。

关键词：图书管理系统；Django；Bootstrap；设计模式

Library management system

Abstract: This system is a popular book management system under the B/S mode, and it is a set of OA system that the library needs to apply, so that managers can easily manage books on the computer at any time. The system architecture design, database design and book management design shall be completed. The main functions are: publishing house management, book management, author management, etc.

On the basis of analyzing the requirements of book management, the system selects PyCharm development environment, MySQL database, Django and Bootstrap framework technologies under Windows 10 operating system, and expounds the working principles and relations of python, Django and other technologies, and realizes the development of background modules such as workbench, user management, publishing house management, author management and book management. MySQL database is selected as the background database, and ORM technology is used to complete the encapsulation and mapping of the database, realizing data query, modification, addition, deletion and other functions. The front-end framework uses the Bootstrap framework, which integrates html, CSS, JavaScript and other front-end languages well. It is a good choice for developers.

Key words: library management system; Django; Bootstrap; Design Mode

WEB 项目实习任务书

一、设计题目

图书管理系统

二、设计背景

在积极建立科学管理机制的今天，仅仅靠原始的手工或者简单的单机管理，管理部门面对大量的信息，无法有效地将其中的重要部分提取出来，并且做相应的判断和处理。图书管理者的决策只能依据与手工表的数据，在浪费大量人力、物力的同时无法做到实时监控，难以保证数据的准确性和及时性。因此先进的管理思想就成为了一个可望而不可及的目标。图书管理非常急需一套既有先进管理思想的系统，作为实现目标和提高现有投票的管理水平的一种重要手段。编写图书管理系统，最重要的一点就是在非常繁琐的图书系统中，将图书进行有序的分类管理，并且最终能够对图书进行增删改查这些管理。

三、设计内容及要求

（一）用户注册登录模块

用户需要有账号才能使用该系统。没有账号的用户需要先进行注册账号。存储账户的信息包括用户的用户名，电子邮箱，电话，等。注册成功后即可通过登录页面登入该系统。

（二）出版社管理模块

出版社管理模块包含出版社编号、出版社名称和出版社地址这几个信息。该模块可以对出版社进行增加、删除、编辑和查找这四个操作。出版社的添加也会影响到后面的模块。

（三）图书管理模块

图书管理模块包含图书序号、书名、ISBN、出版社、作者和出版日期这几个信息。该模块可以对图书进行增加、删除、编辑和查找这四个操作。

（四）作者管理模块

作者管理模块包含序号、作者姓名、作者性别、作者年龄、联系方式和著作这几个信息。该模块可以对作者进行增加、删除、编辑和查找这四个操作。

（五）项目部署

该项目使用 python 语言编写，采用了 python 当中的 Django 框架编写 B/S 应用程序。数据库方面使用 MySQL 数据库，前端框架使用了 Bootstrap。

Web 项目实习（报告）

序号	工 作 内 容	时间安排 (天)
1	任务安排和选题	1
2	前期准备	1
3	需求分析	1
4	系统设计	3
5	编码和调试	4
6	测试和完善	2
7	撰写作业报告	1
8	答辩并且验收作业报告	1
五、设计时间： 2022 年 11 月 28 日 ~ 2022 年 12 月 11 日		

软件工程 系

指导教师 宦娟

1. 前言

1.1 选题背景

在积极建立科学管理机制的今天，仅仅靠原始的手工或者简单的单机管理，管理部门面对大量的信息，无法有效地将其中的重要部分提取出来，并且做相应的判断和处理。图书管理者的决策只能依据与手工表的数据，在浪费大量人力、物力的同时无法做到实时监控，难以保证数据的准确性和及时性。因此先进的管理思想就成为了一个可望而不可及的目标。图书管理非常急需一套既有先进管理思想的系统，作为实现目标和提高现有投票的管理水平的一种重要手段。编写图书管理系统，最重要的一点就是在非常繁琐的图书系统中，将图书进行有序的分类管理，并且最终能够对图书进行增删改查这些管理。

1.2 设计目标

（一）用户注册登录模块

用户需要有账号才能使用该系统。没有账号的用户需要先进行注册账号。存储账户的信息包括用户的用户名，电子邮箱，电话，等。注册成功后即可通过登录页面登入该系统。

（二）出版社管理模块

出版社管理模块包含出版社编号、出版社名称和出版社地址这几个信息。该模块可以对出版社进行增加、删除、编辑和查找这四个操作。出版社的添加也会影响到后面的模块。

（三）图书管理模块

图书管理模块包含图书序号、书名、ISBN、出版社、作者和出版日期这几个信息。该模块可以对图书进行增加、删除、编辑和查找这四个操作。

（四）作者管理模块

作者管理模块包含序号、作者姓名、作者性别、作者年龄、联系方式和著作这几个信息。该模块可以对作者进行增加、删除、编辑和查找这四个操作。

2. 运行环境及开发工具概述

2.1 开发及运行环境概述

硬件要求：

- 1、系统：Windows7 及以上
- 2、内存：4GB 及以上
- 3、CPU：1.8 GHz 或更快的处理器

环境要求：

- 1、语言：Python
- 2、框架：Django
- 3、数据库：MySQL
- 4、前端框架：Bootstrap

2.1.1 Python 简介

Python 由荷兰数学和计算机科学研究学会的吉多·范罗苏姆于 1990 年代初设计，作为一门叫做 ABC 语言的替代品。Python 提供了高效的高级数据结构，还能简单有效地面向对象编程。Python 语法和动态类型，以及解释型语言的本质，使它成为多数平台上写脚本和快速开发应用的编程语言，随着版本的不断更新和语言新功能的添加，逐渐被用于独立的、大型项目的开发。

2.1.2 Django 简介

Django 是高水准的 Python 编程语言驱动的一个开源模型-视图，控制器风格的 Web 应用程序框架，它起源于开源社区。使用这种架构，程序员可以方便、快捷地创建高品质、易维护、数据库驱动的应用程序。这也正是 OpenStack 的 Horizon 组件采用这种架构进行设计的主要原因。另外，在 Django 框架中，还包含许多功能强大的第三方插件，使得 Django 具有较强的可扩展性。

2.1.3 MySQL 简介

MySQL 是一个关系型数据库管理系统，由瑞典 MySQLAB 公司开发，属于 Oracle 旗下产品。MySQL 是最流行的关系型数据库管理系统之一，在 WEB 应用方面，MySQL 是最好的 RDBMS (Relational Database Management System，关系数据库管理系统) 应用软件之一。

2.1.4 Bootstrap 简介

Bootstrap 是美国 Twitter 公司的设计师 Mark Otto 和 Jacob Thornton 合作基于 HTML、CSS、JavaScript 开发的简洁、直观、强悍的前端开发框架，使得 Web 开发更加快捷。Bootstrap 提供了优雅的 HTML 和 CSS 规范，它即是由动态 CSS 语言 Less 写成。Bootstrap 一经推出后颇受欢迎，一直是 GitHub 上的热门开源项目，包括 NASA 的 MSNBC（微软全国广播公司）的 Breaking News 都使用了该项目。国内一些移动开发者较为熟悉的框架，如 WeX5 前端开源框架等，也是基于 Bootstrap 源码进行性能优化而来。

3. 系统的需求分析

3.1 背景和意义

根据各图书销售门店的调查可知，随着销售规模的不断壮大，经营的图书品种、数量也逐渐增多。在图书销售不断发展的同时，其常年采用的传统的人工方式管理暴露了一些问题。

例如，查找读者借阅的某本图书的具体详细信息需要靠人工记忆在书海中苦苦查找，由于图书储存量大，很难准确定位图书的具体位置，因此每天都要浪费大量宝贵的时间资源。

为提高工作效率，同时摆脱图书管理人员在工作中出现的种种弊端，现开发一个图书管理系统。

3.2 可行性分析

通过计算机对图书进行管理，不仅为图书的管理注入了新的生机，而且在运营过程中节省了大量的人力、物力、财力和时间，可以提高图书管理的效率。随着计算机技术的发展，以及计算机在各个企事业单位中应用的普及，计算机强大的功能已为人们深刻认识。它在当今高速发展的信息时代占据着不可缺少的地位，作为计算机应用软件的一部分，使用计算机对图书馆进行综合管理已经远远超过手工管理的效率。因此开发这样一个管理系统成为一件很必要的事情。

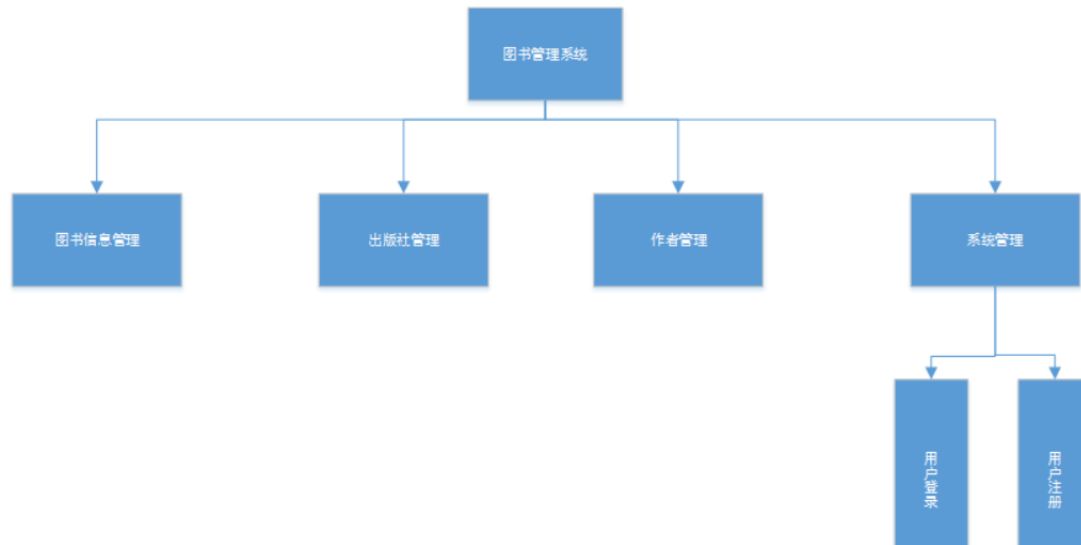
3.3 需求分析

图书管理系统是为了实现图书高效管理而设计的.它完全取代了人工处理的工作方式,并且避免了由于管理员的疏忽以及管理质量问题所导致的各种错误。为及时、准确、高效地完成图书管理工作提供了强有力的工具。

3.3.1 功能需求

本系统主要是管理员对图书信息和出版社进行管理。本系统需要有出版社管理、图书管理、作者管理、统计等功能，能将相关信息从数据库中添加修改删除并且利用查询将相关信息显示出来。数据要求有自动更新功能,能显示最新的结果。

根据学校图书馆管理系统的特点，可以将其分为图书信息管理,出版社管理,作者管理,系统管理等 4 个部分，其中各个部分及其包括的具体功能模块如图所示。



通过对一些大型图书管理系统的实际考察、分析,并结合实际要求以及实际的市场调查,要求本系统具有以下功能:

- 窗体界面要求美观大方、个性化、功能全面、操作简单。
- 要求实现基础信息的管理平台。
- 要求对所有图书进行管理。
- 提供快速的图书信息检索功能,保证数据查询的灵活性。
- 实现综合条件查询,如图书名称、图书作者等。
- 提供灵活、方便的权限设置功能,使整个系统的管理分工明确。
- 具有易维护性和易操作性。

3.3.2 性能需求

因为系统要代替人工管理图书信息,所以要求系统响应速度快,稳定性强,操作简单,准确性高。

根据前面所作的需求分析及功能的需求可以得出,学校图书馆管理系统实施后,应达到以下系统性能标准:

- 窗体界面要求美观大方、功能全面、操作简单。
- 程序整体结构和操作流程合理顺畅,实现人性化设计。
- 规范、完善的基础信息设置。
- 对操作员设置不同的操作权限,为管理员提供修改权限功能。
- 对图书信息进行集中管理。

3.3.3 软件质量需求

对软件运营者而言,要保证有效性.最好易于扩展,有较好的可移植性。对客户而言,要做到操作简单.界面友好,帮助文档充分。

3.3.4 灵活性

当用户需求,如操作方式.运行环境、结果精度、数据结构与其他软件发生变化时,设计的软件要随时做适当的调整。因此,灵活性非常大。

4. 系统设计

4.1 界面设计

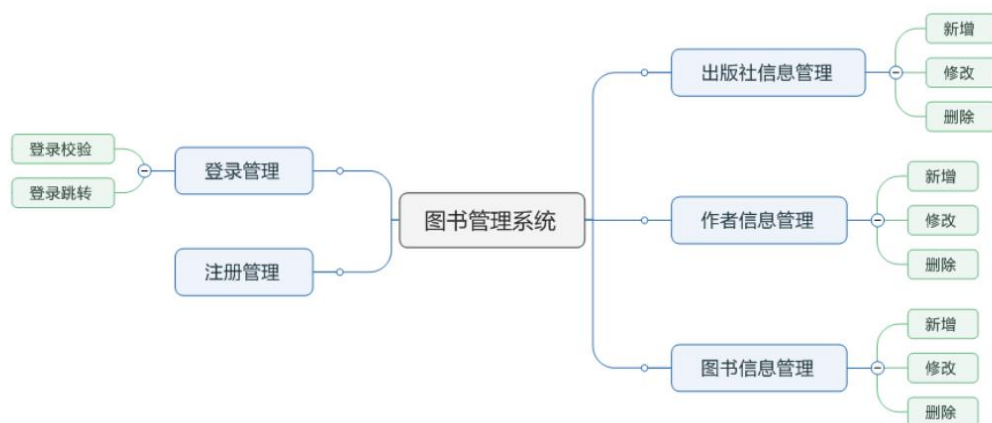
系统界面遵循最小化信息量原则即人机交互的界面设计方案要尽可能的帮助记忆，最大化缓解用户记忆压力；遵循系统用户原则即人机交互的界面的最先要将不同的用户进行分类，不同身份的用户由于权限的限制，可以使用的菜单项也是有差别的；遵循提示与帮助的原则即用户操作时适当的有针对性的做出提示反应，使用户拥有主动权，防止操作失误造成不必要的损失。

数据输入界面是最容易出现差池的部分，也是占用时间最多的部分。为了降低出错率，尽可能的方便用户的使用，减少用户多余的操作，能够极大程度的允许输入及其他操作的误差，可以适当的采用下面的几种设计方式：

- （1）采用列表选择来降低用户的记忆：使用缩写结构及代码，运用默认值的设定等，选择记忆用户之前输入的记录。
- （2）界面要具有统一的数据输入风格。
- （3）用户删除或者是其他的编辑操作需要经过确认，及时做出提示避免致命的失误，可限制输入的字符类型，当输入值为空时有对应的消息提示。
- （4）用户可以随时看到输入的内容，并且能够根据提示准确的进行操作。
- （5）界面在一定的情况下避免大段信息重复敲击输入，可以尽可能自动加载一些信息。

4.2 系统总体设计

根据系统分析的结果，在综合分析的基础上,进行了系统总体设计,把系统分为了 5 个功能模块:登录模块、出版社管理、图书管理、作者管理和系统用户管理模块。系统模块结构图如图所示。



图书信息管理模块:主要包括图书信息管理的功能，具体包括查询图书、修改图书、增加图书、删除图书基本资料等基本操作(普通用户只有查询功能)。

出版社管理模块:主要是对出版社进行管理，可以查询出版社、修改出版社、增加出版社、删

除出版社等功能。

作者管理模块:主要是对作者进行管理,可以查询作者、修改作者、增加作者、删除作者等功能。

登录管理模块:主要是实现用户登录的功能,用户只有登录系统,才能在自己的权限范围内进行合法的操作,否则,将不能进行任何操作。

注册管理模块:主要实现用户注册功能。

4.3 数据库设计

MySQL 具有很强的完整性与可伸缩性,具有较低的价格比与性能比,考虑到本系统的稳定性与可靠性以及开发程序与用户需求,笔者决定在设计该系统时选择 MySQL 数据库来满足系统的需求。

在设计图书馆管理系统之前,有必要说明数据库的基本概念和本设计所应用的数据表。

4.4 系统实体 E-R 图

根据实际调查对系统所做的需求分析、系统设计,规划出本系统中使用的数据库实体分别为图书档案实体、出版社档案实体、作者档案实体、用户档案实体。下面将介绍几个关键实体的 E-R 图。

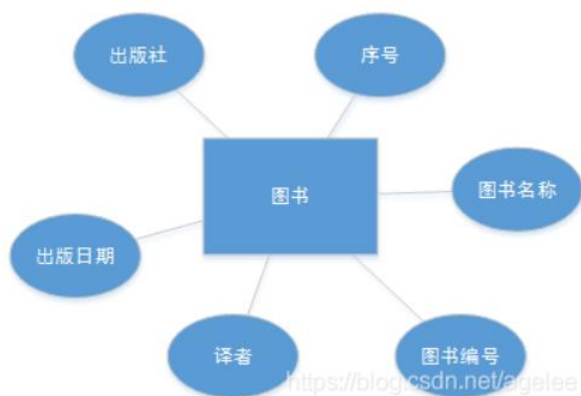
1.出版社档案实体

出版社档案实体包括:出版社编号,出版社地址,出版社名称



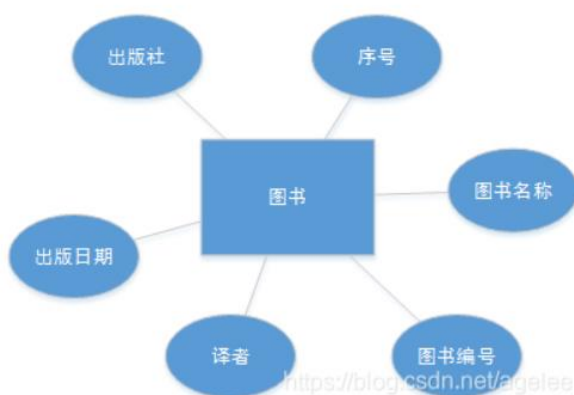
2.图书档案实体

出版社档案实体包括:序号,图书名称,图书编号,译者,出版日期,出版社



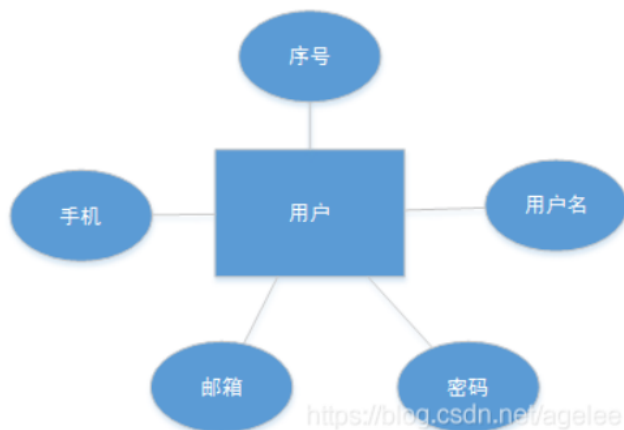
3. 作者档案实体

出版社档案实体包括: 序号, 姓名, 性别, 年龄, 联系方式



4. 用户档案实体

出版社档案实体包括: 序号, 用户名, 密码, 邮箱, 手机



4.5 系统数据表设计

结合实际情况及对用户需求的分析, 图书管理系统 bms 数据库主要包含如下表所示的 4 个数据表。

1. 出版社信息表

字段名	数据类型	是否主键	说明
-----	------	------	----

id	int	是	出版社编号
name	int	否	出版社名称
addr	Varchar(40)	否	出版社地址

2. 图书信息表

字段名	数据类型	是否主键	说明
id	int	是	序号
name	Varchar(64)	否	图书名称
ISBN	Varchar(64)	否	图书编号
translator	Varchar(64)	否	译者
date	date	否	出版日期
publisher_id	int	否	出版社

3. 作者信息表

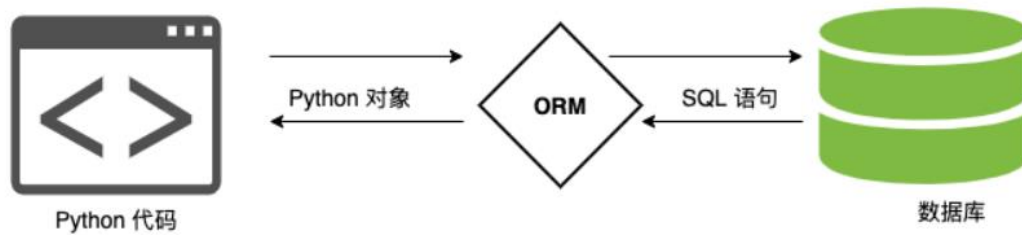
字段名	数据类型	是否主键	说明
id	int	是	序号
name	Varchar(64)	否	姓名
sex	Varchar(64)	否	性别
age	int	否	年龄
tel	Varchar(64)	否	联系方式

4. 用户信息表

字段名	数据类型	是否主键	说明
id	int	是	序号
username	Varchar(64)	否	用户名
password	Varchar(64)	否	密码
email	Varchar(64)	否	邮箱
mobile	int	否	手机

4.6 数据库逻辑设计

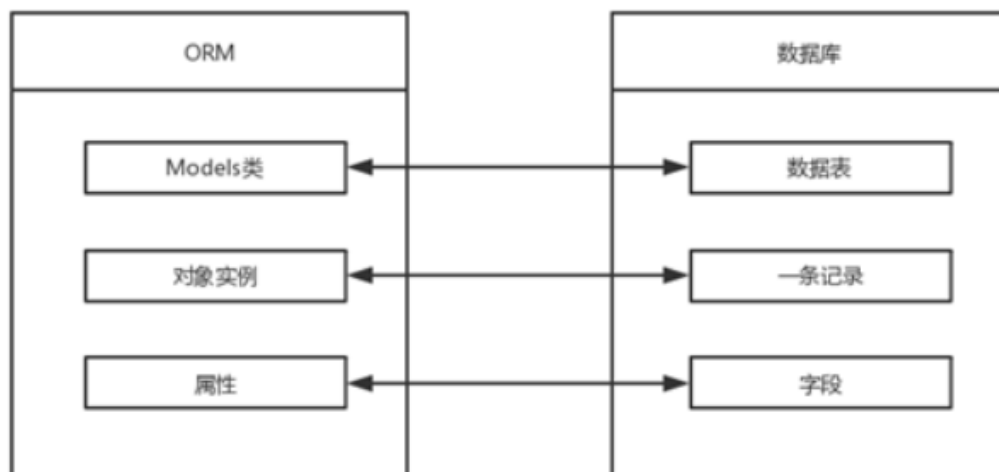
设计数据库使用了 Django 模型使用自带的 ORM。对象关系映射（Object Relational Mapping，简称 ORM）用于实现面向对象编程语言里不同类型系统的数据之间的转换。ORM 在业务逻辑层和数据库层之间充当了桥梁的作用。通过使用描述对象和数据库之间的映射的元数据，将程序中的对象自动持久化到数据库中。



ORM 解析过程:

- 1、ORM 会将 Python 代码转成为 SQL 语句。
- 2、SQL 语句通过 pymysql 传送到数据库服务端。
- 3、在数据库中执行 SQL 语句并将结果返回。

ORM 对应关系表:



5. 图书管理系统的实现

先通过对系统进行需求分析、系统设计等阶段，随之开始进入系统的实施阶段，系统的分析和设计主要着重于逻辑及功能模块的设计，系统的实施是在前期各阶段的工作成果基础之上，物理化功能设计实现整个影院订票系统。结尾附关键代码。

5.1 用户注册登录模块

本模块账户包括用户信息，通过 id 字段区分身份信息，展示不同的模板页面。登录用户密码通过加密存储到 mysql 数据库，每次登录将前端模板页面的 form 表单里的数据获取

到后加密从数据库查询，记录登录信息。

一、首先在 model.py 文件中定义了和用户登录所需的数据库

```
class LmsUser(models.Model):
```

```
    id = models.AutoField('序号', primary_key=True)
```

```
    username = models.CharField('用户名', max_length=32)
```

```
    password = models.CharField('密码', max_length=32)
```

```
    email = models.CharField('邮箱', max_length=254)
```

```
    mobile = models.BigIntegerField('手机', blank=True)
```

通过 django 的 ORM 框架将这个类转换成 MySQL 语句形成数据表。

二、接着在 views.py 文件中定义用户注册和登录方法，方法执行获得用户登陆所需要信息的请求，并且判断输入的用户名密码与数据库中是否一致。并且连接了前端的 html 文件。

```
def login(request):
```

```
    if request.method == 'POST' and request.POST:
```

```
        email = request.POST.get("email")
```

```
        password = request.POST.get("password")
```

```
        e = LmsUser.objects.filter(email = email).first()
```

```
        if e:
```

```
            now_password = setPassword(password)
```

```
            db_password = e.password
```

```
            if now_password == db_password:
```

```
                # return render(request, "pub_list.html")
```

```
                response = HttpResponseRedirect('/pub_list/')
```

```
                response.set_cookie("username", e.username)
```

```
                return response
```

```
    return render(request, "login.html")
```

注册

```
def register(request):
```

```
    if request.method == "POST" and request.POST:
```

```
        data = request.POST
```

```
        username = data.get("username")
```

```
        email = data.get("email")
```

```
        password = data.get("password")
```

```
        mobile = data.get("mobile")
```

```
        LmsUser.objects.create(
```

```
            username=username,
```

```
            email=email,
```

```
            password=setPassword(password),
```

```
            # password=password,
```

```
            mobile=mobile,
```

```
        )
```



```
return HttpResponseRedirect('/login/')
return render(request,"register.html")
```

三、此外，还要编写前端网页来实现用户注册和登录的页面，代码如下：

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>图书管理系统</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- jQuery UI -->
  <link href="https://code.jquery.com/ui/1.10.3/themes/redmond/jquery-ui.css" rel="stylesheet"
media="screen">

  <!-- Bootstrap -->
  <link href="/static/bootstrap/css/bootstrap.min.css" rel="stylesheet">
  <!-- styles -->
  <link href="/static/css/styles.css" rel="stylesheet">

  <!-- HTML5 Shim and Respond.js IE8 support of HTML5 elements and media queries -->
  <!-- WARNING: Respond.js doesn't work if you view the page via file:// -->
  <!--[if lt IE 9]>
  <script src="https://oss.maxcdn.com/libs/html5shiv/3.7.0/html5shiv.js"></script>
  <script src="https://oss.maxcdn.com/libs/respond.js/1.3.0/respond.min.js"></script>
  <![endif]-->
</head>
<body class="login-bg">
  <div class="header">
    <div class="container">
      <div class="row">
        <div class="col-md-12">
          <!-- Logo -->
          <div class="logo">
            <h1><a href="index.html">图书管理系统</a></h1>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div class="page-content container">
    <div class="row">
      <div class="col-md-4 col-md-offset-4">
        <div class="login-wrapper">
```

```
<div class="box">
  <h6>登录</h6>
  <form      class="content-wrap"      role="form"      action="/login/"
method="post">
    {% csrf_token %}
    <input      class="form-control"      name="email"      type="text"
placeholder="E-mail">
    <input class="form-control" name="password" type="password"
placeholder="Password">
    <div class="action">
      <button type="submit" class="btn btn-primary">登录</button>
    </div>
  </form>
</div>
<div class="already">
  <p>没有账户?</p>
  <a href="/register">注册</a>
</div>
</div>
</div>
</div>
</div>

<!-- jQuery (necessary for Bootstrap's JavaScript plugins) -->
<script src="https://code.jquery.com/jquery.js"></script>
<!-- Include all compiled plugins (below), or include individual files as needed -->
<script src="bootstrap/js/bootstrap.min.js"></script>
<script src="js/custom.js"></script>
</body>
</html>
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>图书管理系统</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- jQuery UI -->
  <link href="https://code.jquery.com/ui/1.10.3/themes/redmond/jquery-ui.css" rel="stylesheet"
media="screen">

  <!-- Bootstrap -->
  <link href="/static/bootstrap/css/bootstrap.min.css" rel="stylesheet">
```

```
<!-- styles -->
<link href="/static/css/styles.css" rel="stylesheet">

<!-- HTML5 Shim and Respond.js IE8 support of HTML5 elements and media queries -->
<!-- WARNING: Respond.js doesn't work if you view the page via file:// -->
<!--[if lt IE 9]>
<script src="https://oss.maxcdn.com/libs/html5shiv/3.7.0/html5shiv.js"></script>
<script src="https://oss.maxcdn.com/libs/respond.js/1.3.0/respond.min.js"></script>
<![endif]-->
</head>
<body class="login-bg">
  <div class="header">
    <div class="container">
      <div class="row">
        <div class="col-md-12">
          <!-- Logo -->
          <div class="logo">
            <h1><a href="index.html">图书管理系统</a></h1>
          </div>
        </div>
      </div>
    </div>

    <div class="page-content container">
      <div class="row">
        <div class="col-md-4 col-md-offset-4">
          <div class="login-wrapper">
            <div class="box">
              <form class="content-wrap" role="form" action="/register/"
method="post">
                <h6>注册</h6>
                {% csrf_token %}
                <input class="form-control" name="username" type="text"
placeholder="Username">
                <input class="form-control" name="email" type="text"
placeholder="E-mail address">
                <input class="form-control" name="password" type="password"
placeholder="Password">
                <input class="form-control" name="mobile" type="text"
placeholder="Mobile">
                <!--
                <input class="form-control" type="password"
placeholder="Confirm Password">-->
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

四、最后在 `urls.py` 文件中配置路由 `url` 和视图 `view` 中函数的映射关系

```
url(r'^signup/', views.register),      # 注册页面
url(r'^register/', views.register),    # 注册
```

注册

Username

E-mail address

Password

Mobile

注册

已有账户? [登录](#)

登录

E-mail

Password

登录

没有账户? [注册](#)

5.2 出版社管理模块

本模块是实现对出版社增、删、改、查这四个操作的模块，在 `models.py` 文件中定义了 `publisher` 类，类中有 `id`, `name`, `address` 等对象。通过运用 Django 中 ORM 映射来根据这个类生成对应的 SQL 语句，从而产生相应的数据库。在 `views.py` 中定义了 `publisher_list`, `add_publisher`, `edit_publisher` 和 `drop_publisher` 这些函数。模块前端的实现是由这些 HTML, CSS 和 JavaScript 写成的 `pub_list`, `pub_add` 和 `pub_edit` 的前端代码，最后在 `urls.py` 中定义了 url 和前端代码的映射。

The image displays three screenshots of a web application titled '图书管理系统' (Library Management System). Each screenshot shows a sidebar with navigation options: '出版社管理' (Publisher Management), '作者管理' (Author Management), '图书管理' (Book Management), and '系统设置' (System Settings). The top navigation bar includes a search bar and a user profile icon.

Top Screenshot: Publisher List
The main content area shows a table titled '出版社列表' (Publisher List). The table has columns for '序号' (Serial Number), '出版社名称' (Publisher Name), '出版社地址' (Publisher Address), and '操作' (Actions). The table contains two entries:

序号	出版社名称	出版社地址	操作
1	清华大学出版社	清华大学	编辑 删除
2	北京大学出版社	北京大学	编辑 删除

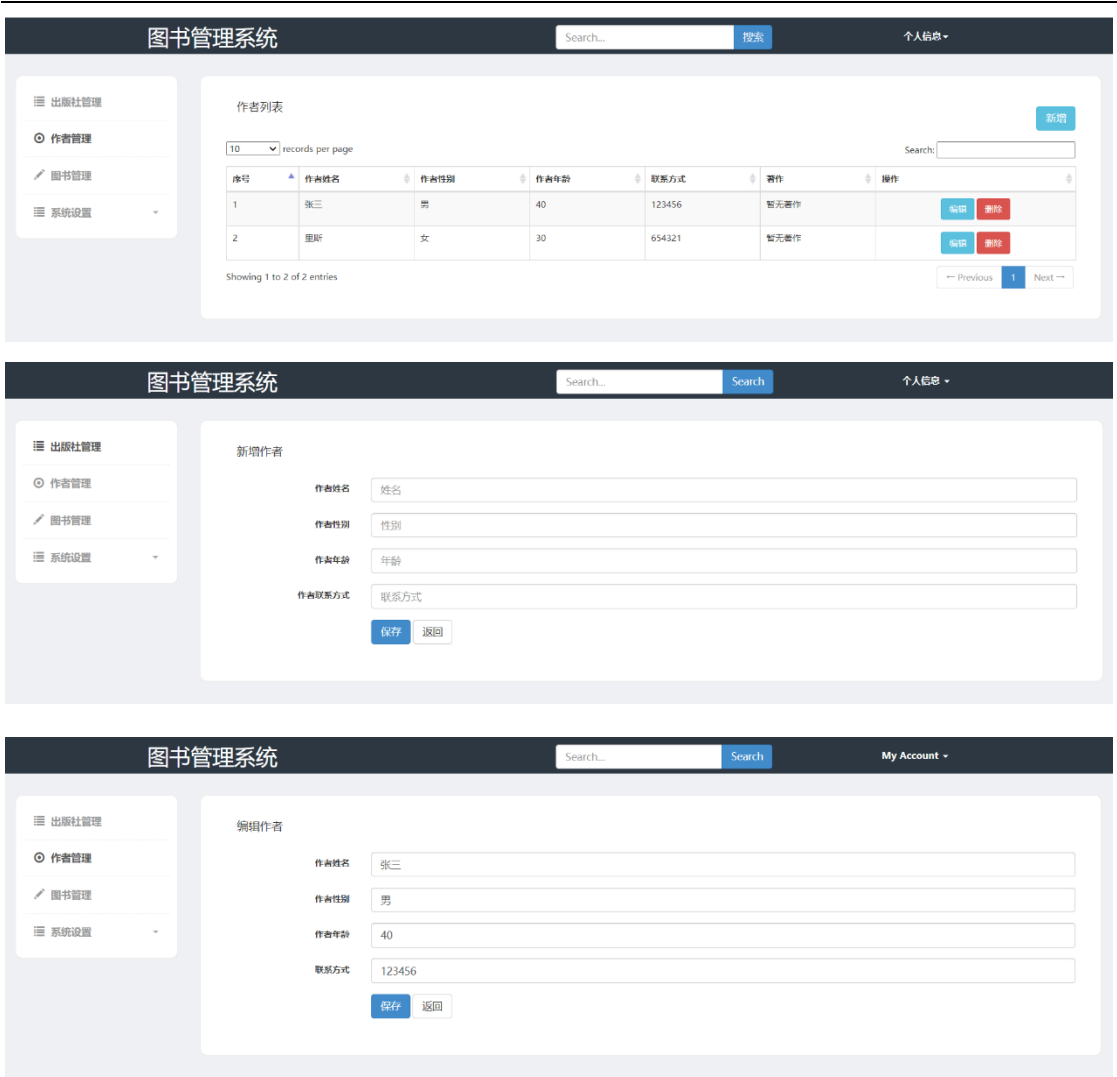
Below the table, it says 'Showing 1 to 2 of 2 entries'. There are pagination controls for 'Previous', '1', and 'Next'. A '新增' (Add) button is in the top right corner.

Middle Screenshot: Add Publisher
The main content area shows a form titled '新增出版社' (Add Publisher). It has two input fields: '出版社名称' (Publisher Name) and '出版社地址' (Publisher Address). At the bottom, there are '保存' (Save) and '返回' (Return) buttons.

Bottom Screenshot: Edit Publisher
The main content area shows a form titled '编辑出版社' (Edit Publisher). It has two input fields: '出版社名称' (Publisher Name) and '出版社地址' (Publisher Address). At the bottom, there are '保存' (Save) and '返回' (Return) buttons.

5.3 作者管理模块

本模块是实现对出作者增、删、改、查这四个操作的模块，在 `models.py` 文件中定义了 `Author` 类，类中有 `id`, `name`, `age`, `sex`, `mobile` 等对象。通过运用 Django 中 ORM 映射来根据这个类生成对应的 SQL 语句，从而产生相应的数据库。在 `views.py` 中定义了 `author_list`, `add_author`, `edit_author` 和 `drop_author` 这些函数。模块前端的实现是由这些 HTML, CSS 和 JavaScript 写成的 `auth_list`, `author_add` 和 `auth_edit` 的前端代码，最后在 `urls.py` 中定义了 url 和前端代码的映射。



5.4 图书管理模块

本模块是实现对图书增、删、改、查这四个操作的模块，在 `models.py` 文件中定义了 `Book` 类，类中有 `id`, `name`, `ISBN`, `translator`, `date`, `publisher` 等对象。通过运用 Django 中 ORM 映射来根据这个类生成对应的 SQL 语句，从而产生相应的数据库。在 `views.py` 中定义了 `book_list`, `add_book`, `edit_book` 和 `drop_book` 这些函数。模块前端的实现是由这些 HTML, CSS 和 JavaScript 写成的 `book_list`, `book_add` 和 `book_edit` 的前端代码，最后在 `urls.py` 中定义了 url 和前端代码的映射。

Web 项目实习（报告）

图书管理系统

Search...

Search

个人信息

出版社管理

作者管理

图书管理

系统设置

编辑书籍

书名张二的自传

ISBN654321

书籍列表清华大学出版社

译者里斯

出版日期April 27, 2023

保存返回

图书管理系统

Search...

Search

个人信息

出版社管理

作者管理

图书管理

系统设置

书籍列表

10 records per page

Search:

序号	书名	ISBN	出版社	译者	出版日期	操作
1	Java程序设计	12345555	清华大学出版社	王五	April 27, 2021	编辑删除
2	数据库原理	12345556	清华大学出版社	王五	Dec. 5, 2022	编辑删除
3	张二的自传	654321	清华大学出版社	里斯	April 27, 2023	编辑删除

Showing 1 to 3 of 3 entries

Previous1Next

图书管理系统

Search...

Search

个人信息

出版社管理

作者管理

图书管理

系统设置

新增书籍

书名

ISBN

出版社名称清华大学出版社

译者译者

出版日期2021-04-27

保存返回

5.5 关键代码

```
import hashlib
```

```
from django.http import HttpResponseRedirect
from django.shortcuts import render, redirect
from django.views.decorators.csrf import csrf_exempt
```

```
from app01 import models
```

```
# Create your views here.
```

```
# 出版社展示列表
from app01.models import LmsUser
```

```
def publisher_list(request):
    publisher = models.Publisher.objects.all()
    return render(request, 'pub_list.html', {'pub_list': publisher})
```

添加出版社

```
def add_publisher(request):
    if request.method == 'POST':
        new_publisher_name = request.POST.get('name')
        new_publisher_addr = request.POST.get('addr')
        models.Publisher.objects.create(name=new_publisher_name, addr=new_publisher_addr)
        return redirect('/pub_list/')
    return render(request, 'pub_add.html')
```

编辑出版社

```
def edit_publisher(request):
    if request.method == 'POST':
        edit_id = request.GET.get('id')
        edit_obj = models.Publisher.objects.get(id=edit_id)
        new_name = request.POST.get('edit_name')
        new_addr = request.POST.get('edit_addr')
        edit_obj.name = new_name
        edit_obj.addr = new_addr
        edit_obj.save()
        return redirect('/pub_list/')

    edit_id = request.GET.get('id')
    edit_obj = models.Publisher.objects.get(id=edit_id)
    return render(request, 'pub_edit.html', {'publisher': edit_obj})
```

删除出版社

```
def drop_publisher(request):
    drop_id = request.GET.get('id')
    drop_obj = models.Publisher.objects.get(id=drop_id)
    drop_obj.delete()
    return redirect('/pub_list/')
```


作者的列表

```
def author_list(request):
    author = models.Author.objects.all()
    return render(request, 'auth_list.html', {'author_list': author})
```

添加作者

```
def add_author(request):
    if request.method == 'POST':
        new_author_name = request.POST.get('name')
        new_author_sex = request.POST.get('sex')
        new_author_age = request.POST.get('age')
        new_author_tel = request.POST.get('tel')
        models.Author.objects.create(name=new_author_name, sex=new_author_sex,
age=new_author_age, tel=new_author_tel)
        return redirect('/author_list/')
    return render(request, 'author_add.html')
```

删除作者

```
def drop_author(request):
    drop_id = request.GET.get('id')
    drop_obj = models.Author.objects.get(id=drop_id)
    drop_obj.delete()
    return redirect('/author_list/')
```

修改作者

```
def edit_author(request):
    if request.method == 'POST':
        edit_id = request.GET.get('id')
        edit_obj = models.Author.objects.get(id=edit_id)
        new_author_name = request.POST.get('edit_name')
        new_author_sex = request.POST.get('edit_sex')
        new_author_age = request.POST.get('edit_age')
        new_author_tel = request.POST.get('edit_tel')
        new_book_id = request.POST.getlist('book_id')
        edit_obj.name = new_author_name
        edit_obj.sex = new_author_sex
        edit_obj.age = new_author_age
        edit_obj.tel = new_author_tel
        edit_obj.book.set(new_book_id)
        edit_obj.save()
```

```
        return redirect('/author_list/')
    edit_id = request.GET.get('id')
    edit_obj = models.Author.objects.get(id=edit_id)
    all_book = models.Book.objects.all()
    return render(request, 'auth_edit.html', {
        'author': edit_obj,
        'book_list': all_book
    })

# 书籍列表
def book_list(request):
    book = models.Book.objects.all()
    return render(request, 'book_list.html', {'book_list': book})

# 添加书籍
def add_book(request):
    if request.method == 'POST':
        new_book_name = request.POST.get('name')
        new_book_ISBN = request.POST.get('ISBN')
        new_book_translator = request.POST.get('translator')
        new_book_date = request.POST.get('date')
        publisher_id = request.POST.get('publisher_id')
        models.Book.objects.create(name=new_book_name,
                                   publisher_id=publisher_id,
                                   ISBN=new_book_ISBN,
                                   translator=new_book_translator, date=new_book_date)
        return redirect('/book_list/')
    res = models.Publisher.objects.all()
    return render(request, 'book_add.html', {'publisher_list': res})

# 删除书籍
def drop_book(request):
    drop_id = request.GET.get('id')
    drop_obj = models.Book.objects.get(id=drop_id)
    drop_obj.delete()
    return redirect('/book_list/')

# 编辑书籍
def edit_book(request):
    if request.method == 'POST':
        new_book_name = request.POST.get('name')
        new_book_ISBN = request.POST.get('ISBN')
```

```
new_book_translator = request.POST.get('translator')
new_book_date = request.POST.get('date')
new_publisher_id = request.POST.get('publisher_id')
edit_id = request.GET.get('id')
edit_obj = models.Book.objects.get(id=edit_id)
edit_obj.name = new_book_name
edit_obj.ISBN = new_book_ISBN
edit_obj.translator = new_book_translator
edit_obj.date = new_book_date
edit_obj.publisher_id = new_publisher_id
edit_obj.save()
return redirect('/book_list/')
edit_id = request.GET.get('id')
edit_obj = models.Book.objects.get(id=edit_id)
all_publisher = models.Publisher.objects.all()
return render(request, 'book_edit.html', {'book': edit_obj, 'publisher_list': all_publisher})
```

密码加密

```
def setPassword(password):
```

```
    """
```

```
    加密密码，算法单次md5
```

```
    :param apssword: 传入的密码
```

```
    :return: 加密后的密码
```

```
    """
```

```
    md5 = hashlib.md5()
```

```
    md5.update(password.encode())
```

```
    password = md5.hexdigest()
```

```
    return str(password)
```

登录

```
def login(request):
```

```
    if request.method == 'POST' and request.POST:
```

```
        email = request.POST.get("email")
```

```
        password = request.POST.get("password")
```

```
        e = LmsUser.objects.filter(email = email).first()
```

```
        if e:
```

```
            now_password = setPassword(password)
```

```
            db_password = e.password
```

```
            if now_password == db_password:
```

```
                # return render(request, "pub_list.html")
```

```
                response = HttpResponseRedirect('/pub_list/')
```

```
                response.set_cookie("username", e.username)
```

```
        return response

    return render(request, "login.html")

# 注册

def register(request):
    if request.method == "POST" and request.POST:
        data = request.POST
        username = data.get("username")
        email = data.get("email")
        password = data.get("password")
        mobile = data.get("mobile")
        LmsUser.objects.create(
            username=username,
            email=email,
            password=setPassword(password),
            # password=password,
            mobile=mobile,
        )
        return HttpResponseRedirect('/login/')
    return render(request, "register.html")
```

6. 系统的调试与测试

本系统测试用例如表所示。

测试编号	测试点	动作	数据	预期结果
1	进入系统	点击首页标题图书管理系统		跳转到主界面
2 3	用户登录 用户注册	分别在【e-mail】和【密码】框中输入用户名和密码，点击【登陆】	账号： admin@qq.com 密码： 123456	登陆成功，进入系统主页面

		点击【注册?】，在注册页面输入相应的信息，点击【注册】	username: zy e-mail:admin@qq.com password:123456 mobile:1234567890	注册成功，返回登陆界面，
		分别在【e-mail】和【密码】框中输入用户名和密码，点击【登陆】	账号: admin@qq.com 密码: 123456	登陆成功，进入系统主页面
4	出版社管理	点击【新增】，在跳转的页面输入要添加的数据	出版社名称: 常州大学出版社 出版社地址: 常州大学	新增出版社成功，跳转至出版社管理界面，出版社管理列表显示出新增的类别
		在【查询输入框】内输入需要查询的内容，按下回车	出版社名称: 常州大学	查询出版社内容成功
		点击【编辑图标】，在跳转的页面中输入要修改的数据	出版社地址: 常州大学武进校区	修改出版社内容成功，跳转出版社管理界面，出版社管理记录显示的是修改后的数据
		点击的【删除图标】	出版社名称: 常州大学	删除出版社成功

5	作者管理	点击【作者管理】，再点击【新增】，在跳转的页面内输入要添加的数据	作者姓名：张三 作者性别：男 作者年龄：40 作者联系方式：11111	新增作者成功，跳转至作者管理界面，作者列表显示出新增的作者
		在【查询输入框】内输入需要查询的内容，按下回车	作者姓名：张三	查询成功，作者列表界面只出现查询的作者

		点击【编辑图标】， 在输入框中输入 要修改的数据	作者性别：女	修改作者成功，跳 转作者管理列表 界面，记录显示的 是修改后的数据
		点击右侧的【删除 图标】	作者名称：张三	删除作者成功
6	图书管理	点击【新增】， 在跳转的页面输 入要添加的数据	书名：c 语言程序设计 ISBN：307564892 出版社名称：清华大学出版社 译者：李四 出版日期：2022-12-7	新增图书成功， 跳转至图书管理 界面，图书管理 列表显示出新增 的类别
		在【查询输入 框】内输入需要 查询的内容，按 下回车	书名：C 语言程序设计	查询图书内容成 功

		点击【编辑图标】，在跳转的页面中输入要修改的数据	出版社名称：北京大学出版社	修改图书内容成功，跳转图书管理界面，图书管理记录显示的是修改后的数据
		点击的【删除图标】	书名：C 语言程序设计	删除图书成功

7. 总结

在互联网+的大数据时代，影院订票系统能满足管理者需求。从最初的需求分析、系统设计到最后的系统实现，全都是从用户出发，站在管理者的角度考虑，从而达到了与众不同的图书管理系统。在开发系统的过程中，引用了 B/S 三层结构技术，利用了 Python 语言使用简单界面友好，只需编译一次无需重复这一特点，结合了 Django 和 BooStrap 这一复合框架。

图书管理系统通过可行性分析，需求分析，系统设计，系统实现，系统测试来设计与实现的。首先进行可行性分析，提出了运用的技术。在需求分析时，我们进行了功能需求分析和性能需求分析，功能需求分析中列出了用户和管理员在该系统中进行的操作。接着进行系统的设计，主要是业务流程的设计和模块图的设计，要从实际出发，分析影院订票系统主要有什么用，什么人使用，从而设计出符合实际的功能模块图来，然后进行数据库的设计与 E-R 图的实现。逻辑结构设计的主要工作是把概念结构设计中得到的概念模型转换为具体 DBMS 支持的数据模型，并且会对其进行优化和完善。在进行系统实现时主界面非常重要，主界面是在运行此软件时运用最多的界面，所以此窗口的设计应该尽可能的简介美观。设计的好坏将直接影响到整个应用程序的基本形象，而且主界面是本应用程序的一个控制核心。同时要设计出合理的其他界面。当系统实现后进行系统测试，系统测试的软件测试的目的不是为了测试软件已经完美，而是为了测试软件中隐藏的错误从而对软件进行进一步完善，系统测试主要知道系统的测试进度表，编写出合理的测试用例，最后总结出测试结果。

参考文献

- [1]曹刘阳.编写高质量代码-Web 前端开发修炼之道[M].北京:机械工业出版社,2014.
- [2]李俊青,季文天.高职 WEB 程序设计实训课程探索与实践[J].职业教育研究,2019
- [3]曹刘阳.编写高质量代码-Web 前端开发修炼之道[M].北京:机械工业出版社,2018.
- [4]杨岢,吴天吉.基于 Aspnet 的网站开发前端技术优化研究[J].软件导刊,2013,12(5).